Dbt 05 Materializations

Source originale : https://docs.getdbt.com/docs/build/materializations

Materializations

are strategies for persisting dbt models in a warehouse. There are five types of materializations built into dbt. They are:

table

view

incremental

ephemeral

materialized view

You can also configure

custom materializations

in dbt. Custom materializations are a powerful way to extend dbt's functionality to meet your specific needs.

For a detailed guide on materializations, refer to

Materializations best practices

. For information about data streaming, refer to

How to handle real-time data

.

Learn by video!

For video tutorials on

Materializations

, go to dbt Learn and check out the

Materializations fundamentals

course

.

By default, dbt models are materialized as "views". Models can be configured with a different materialization by supplying the

materialized

configuration

parameter as shown in the following tabs.

Project file

Model file

Property file

dbt_project.yml

The following dbt_project.yml configures a project that looks like this:

.

■■■ models

■■■ csvs

■ ■■■ employees.sql

■ ■■■ goals.sql

■■■ events

■■■ stg_event_log.sql

■■■ stg_event_sessions.sql

name

:

my_project

version

:

1.0.0

config-version

:

2

models

:

my_project

:

events

:

materialize all models in models/events as tables

+materialized

:

table

csvs

:

this is redundant, and does not need to be set

+materialized

:

view

Alternatively, materializations can be configured directly inside of the model SQL files. This can be useful if you are also setting [Performance Optimization] configs for specific models (for example,

Redshift specific configurations
or
BigQuery specific configurations
).
models/events/stg_event_log.sql
{{ config
(
materialized
=
'table'
,
sort
=
'timestamp'
,
dist
=
'user_id'
)
}}
select
*
from
.
Materializations can also be configured in the model's
properties.yml
file. The following example shows the
table
materialization type. For a complete list of materialization types, refer to
materializations
.
models/properties.yml
models
:
-
name

:

events

config

:

materialized

:

table

When using the

view

materialization, your model is rebuilt as a view on each run, via a

create view as

statement.

Pros:

No additional data is stored, views on top of source data will always have the latest records in them.

Cons:

Views that perform a significant transformation, or are stacked on top of other views, are slow to query.

Advice:

Generally start with views for your models, and only change to another materialization when you notice performance problems.

Views are best suited for models that do not do significant transformation, for example, renaming, or recasting columns.

When using the

table

materialization, your model is rebuilt as a

table

on each run, via a

create table as

statement.

Pros:

Tables are fast to query

Cons:

Tables can take a long time to rebuild, especially for complex transformations

New records in underlying source data are not automatically added to the table

Advice:

Use the table materialization for any models being queried by BI tools, to give your end user a faster experience

Also use the table materialization for any slower transformations that are used by many downstream models

incremental

models allow dbt to insert or update records into a table since the last time that model was run.

Pros:

You can significantly reduce the build time by just transforming new records

Cons:

Incremental models require extra configuration and are an advanced usage of dbt. Read more about using incremental models

here

.

Advice:

Incremental models are best for event-style data

Use incremental models when your

dbt run

s are becoming too slow (i.e. don't start with incremental models)

ephemeral

models are not directly built into the database. Instead, dbt will interpolate the code from an ephemeral model into its dependent models using a common table expression (

CTE

). You can control the identifier for this CTE using a

model alias

, but dbt will always prefix the model identifier with

__dbt__cte__

.

Pros:

You can still write reusable logic

Ephemeral models can help keep your

data warehouse

clean by reducing clutter (also consider splitting your models across multiple schemas by

using custom schemas

).

Cons:

You cannot select directly from this model.

Operations

(for example, macros called using

dbt run-operation

cannot

ref()

ephemeral nodes)

Overuse of ephemeral materialization can also make queries harder to debug.

Ephemeral materialization doesn't support

model contracts

.

Advice:

Use the ephemeral materialization for:

Very light-weight transformations that are early on in your DAG

Are only used in one or two downstream models, and

Don't need to be queried directly

The

materialized_view

materialization allows the creation and maintenance of materialized views in the target database.

Materialized views are a combination of a view and a table, and serve use cases similar to incremental models.

Pros:

Materialized views combine the query performance of a table with the data freshness of a view

Materialized views operate much like incremental materializations, however they are usually

able to be refreshed without manual interference on a regular cadence (depending on the database), forgoing the regular dbt batch refresh

required with incremental materializations

dbt run

on materialized views corresponds to a code deployment, just like views

Cons:

Due to the fact that materialized views are more complex database objects, database platforms tend to have

fewer configuration options available; see your database platform's docs for more details

Materialized views may not be supported by every database platform

Advice:

Consider materialized views for use cases where incremental models are sufficient, but you would like the data platform to manage the incremental logic and refresh.

This materialization makes use of the

on_configuration_change

config, which aligns with the incremental nature of the namesake database object. This setting tells dbt to attempt to

make configuration changes directly to the object when possible, as opposed to completely recreating the object to implement the updated configuration. Using dbt-postgres as an example, indexes can be dropped and created on the materialized view without the need to recreate the materialized view itself.

In the context of a dbt run command, materialized views should be thought of as similar to views. For example, a dbt run command is only needed if there is the potential for a change in configuration or sql; it's effectively a deploy action. By contrast, a dbt run command is needed for a table in the same scenarios AND when the data in the table needs to be updated.

This also holds true for incremental and snapshot models, whose underlying relations are tables. In the table cases, the scheduling mechanism is either dbt or your local scheduler; there is no built-in functionality to automatically refresh the data behind a table.

However, most platforms (Postgres excluded) provide functionality to configure automatically refreshing a materialized view.

Hence, materialized views work similarly to incremental models with the benefit of not needing to run dbt to refresh the data.

This assumes, of course, that auto refresh is turned on and configured in the model.

info

dbt-snowflake does not support materialized views, it uses Dynamic Tables instead. For details, refer to Snowflake specific configurations.

Beginning in v1.13, dbt-snowflake supports interactive tables, which are optimized for low-latency, interactive queries and can refresh automatically like dynamic tables. For details, refer to

Interactive tables

.

Python models support two materializations:

table

incremental

Incremental Python models support all the same

incremental strategies

as their SQL counterparts. The specific strategies supported depend on your adapter.

Python models can't be materialized as

view

or

ephemeral

. Python isn't supported for non-model resource types (like tests and snapshots).

For incremental models, like SQL models, you will need to filter incoming tables to only new rows of data:

Snowpark

PySpark

models/my_python_model.py

import

snowflake

.

snowpark

.

functions

as

F

def

model

(

dbt

,

session

)

:

dbt

.

config

(

materialized

=

"incremental"

)

df

=

dbt

.

ref

(

"upstream_table"

)

if

dbt

.

is_incremental

:

only new rows compared to max in current table

max_from_this

=

f"select max(updated_at) from

{

dbt

.

this

}

"

df

=

df

.

filter

(

df

.

updated_at
>=
session
.
sql
(
max_from_this
)
.
collect
(
)
[
0
]
[
0
]
)
or only rows from the past 3 days
df
=
df
.
filter
(
df
.
updated_at
>=
F
.
dateadd
(
"day"
,

F
.
lit
(
-
3
)
,
F
.
current_timestamp
(
)
.
return
df
models/my_python_model.py
import
pyspark
.
sql
.
functions
as
F
def
model
(
dbt
,
session
)
:
dbt
.
config

(

materialized

=

"incremental"

)

df

=

dbt

.

ref

(

"upstream_table"

)

if

dbt

.

is_incremental

:

only new rows compared to max in current table

max_from_this

=

f"select max(updated_at) from

{

dbt

.

this

}

"

df

=

df

.

filter

(

df

.

```
updated_at
>=
session
.
sql
(
max_from_this
)
.
collect
(
)
[
0
]
[
0
]
)
# or only rows from the past 3 days
df
=
df
.
filter
(
df
.
updated_at
>=
F
.
date_add
(
F
.
```

current_timestamp

(

)

,

F

.

lit

(

-

3

)

.

return

df

Note:

Incremental models are supported on BigQuery/Dataproc for the

merge

incremental strategy. The

insert_overwrite

strategy is not yet supported.

0